EXPERIMENT 13: PRO-CODE EXTENSION USING ZOHO CATALYST

AIM

To architect, deploy, and integrate a serverless Pro-Code microservice using Zoho Catalyst (Function-as-a-Service) to extend the functional compute capabilities of the Zoho Creator low-code application 'Vehicle Service & Repair Request Management'. The objective is to decouple complex algorithmic cost calculations and business logic from low-code execution governor caps and bridge them via secured HTTP REST invocation endpoints.

APPLICATION CONTEXT & SYSTEM OVERVIEW

While low-code platforms provide rapid user interface modeling and standard record management, they are bounded by execution statement caps, limited third-party algorithmic library availability, and execution timeouts within native script interpreters (Deluge). Zoho Catalyst serves as an enterprise serverless Pro-Code backend (FaaS) that provides full-stack cloud runtime environments (Node.js, Java, Python). By offloading intensive compute operations—such as multi-variable mechanical labor cost indexing, tiered component taxations, and parts margin algorithms—to Catalyst serverless functions, the enterprise achieves an optimal hybrid architecture that maintains low-code front-end agility backed by pro-code computational scalability.

PROCEDURE

Step 1: Accessing the Zoho Catalyst Cloud Developer Platform

Navigate to the Zoho Catalyst enterprise portal. Verify platform availability and launch the serverless developer suite via the 'Access Catalyst' workbench to initialize full-stack compute, serverless APIs, and managed microservices.

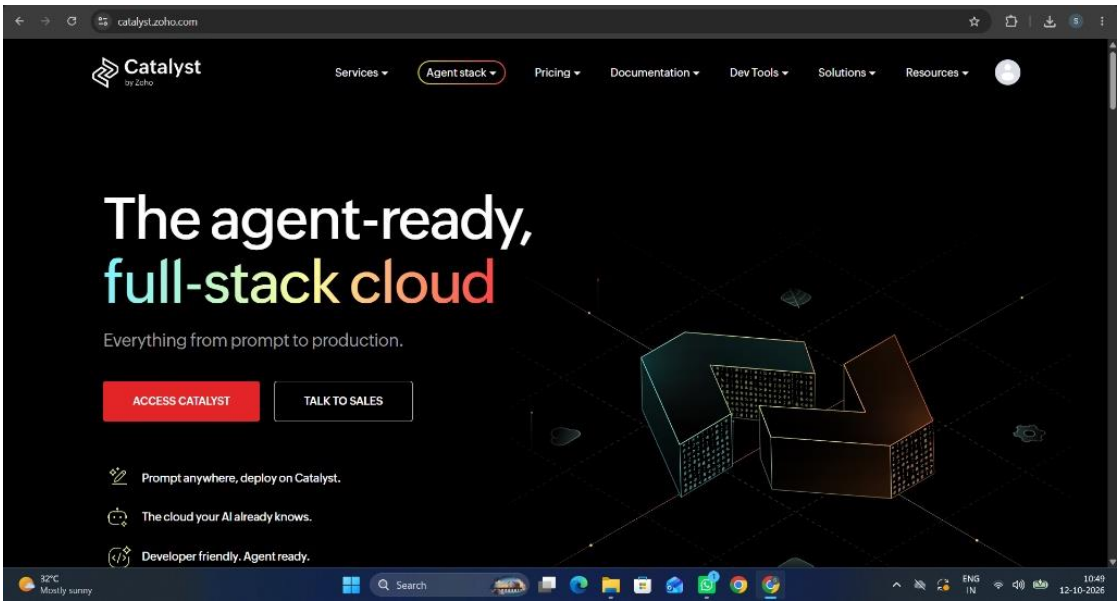


Figure 13.1: Zoho Catalyst Enterprise Full-Stack Cloud Workbench

Step 2: Provisioning Development Workspace in Project Console

Initialize the developer project workspace ('Project-Rainfall') within the Catalyst BaaS console. Verify the administrative tenant hierarchy and access the platform navigation drawer containing Serverless, Cloud Scale, Zia AI engines, and Job Schedulers.

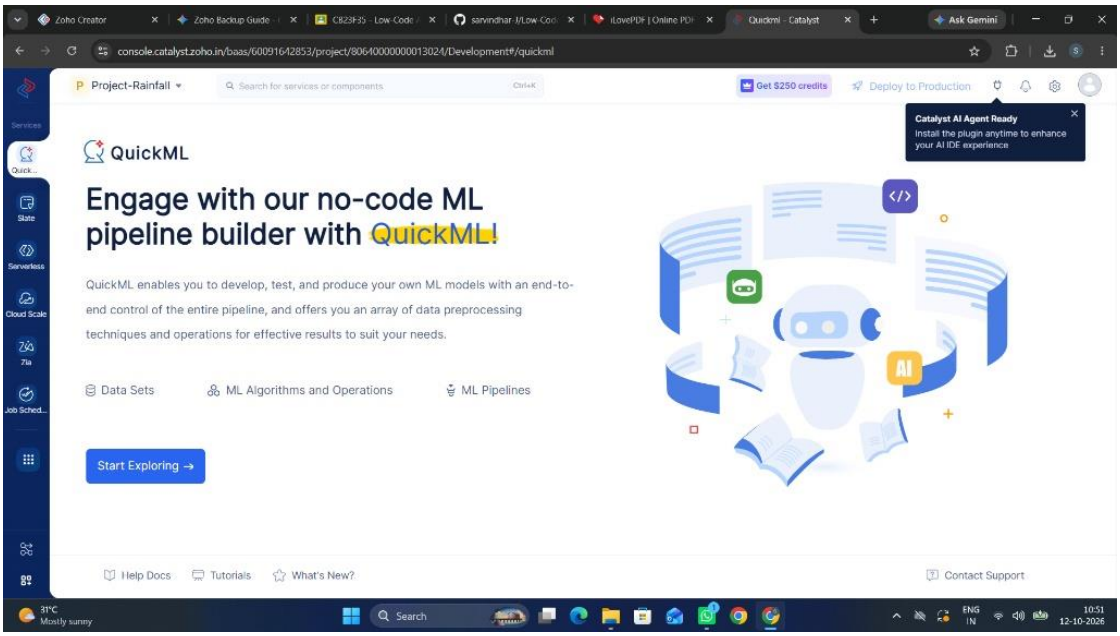


Figure 13.2: Project Workspace Dashboard and Service Component Canvas

Step 3: Initializing Serverless Compute and FaaS Infrastructure

From the left navigation drawer, select 'Serverless' and open the FaaS (Function-as-a-Service) console. The operational workbench displays execution telemetry (Synchronous/Asynchronous Executions, Component Calls, and Cron Invocations) along with Catalyst CLI terminal configurations.

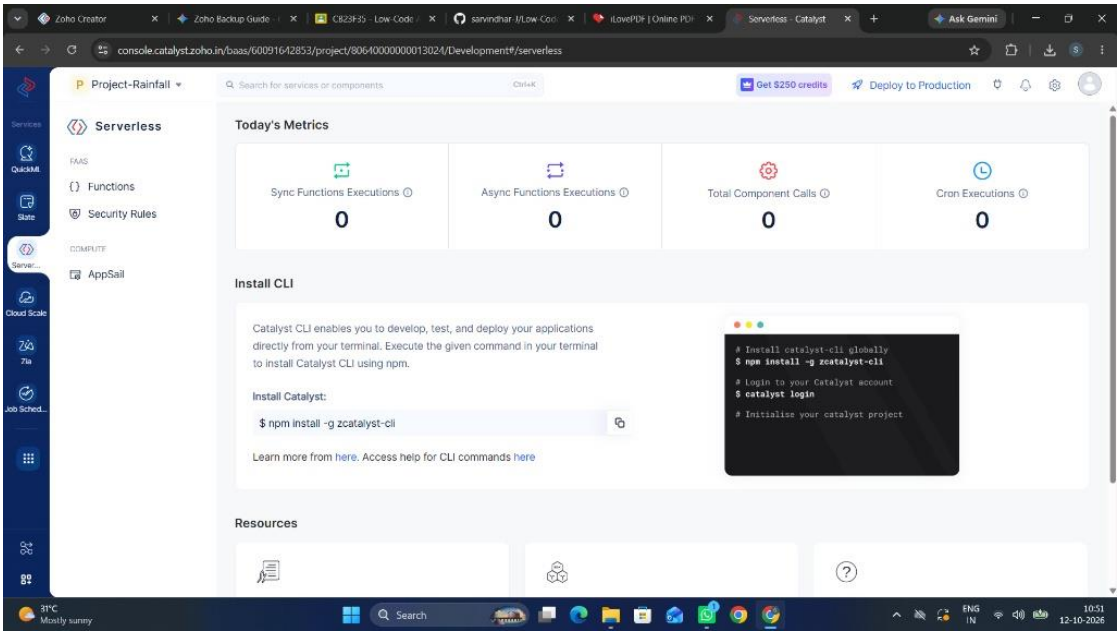


Figure 13.3: Serverless FaaS Telemetry and CLI Orchestration Workbench

Step 4: Architecting Serverless Function Configuration and Runtime Stack

Under the FAAS menu, select 'Functions' and initiate 'Create Function'. Choose the Web Editor method and configure the serverless function parameters: select Function Type as 'Basic IO', define Function Name as 'CalculateServiceCost', and select the runtime execution stack (Node.js 24).

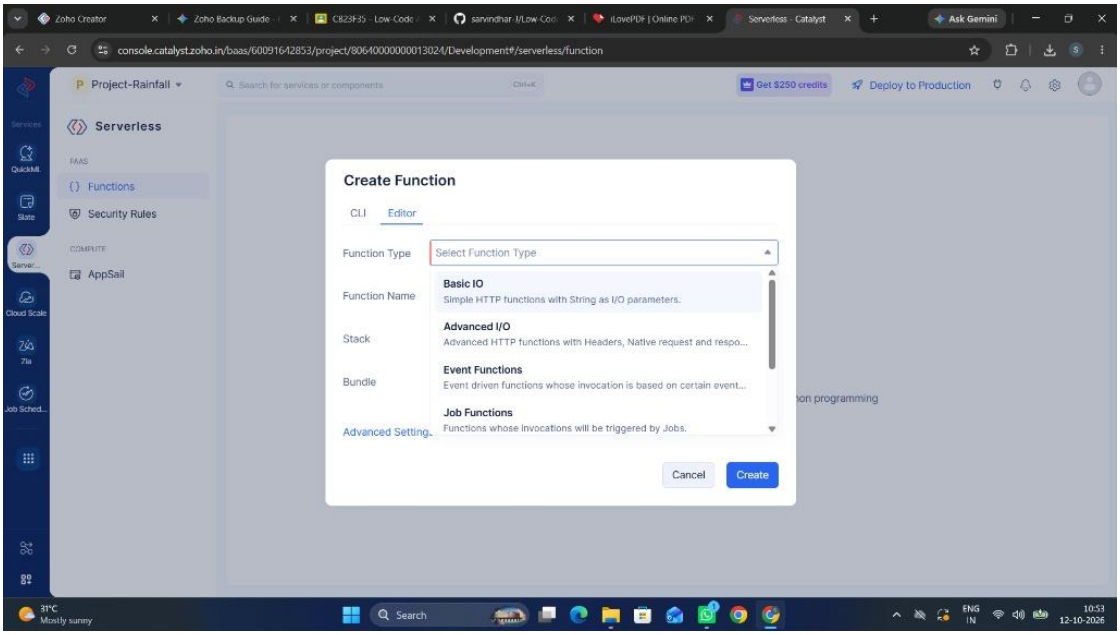


Figure 13.4: Serverless Function Specification and Node Runtime Configuration

Step 5: Inspecting Function Overview, ID Provisioning, and Invocation URL

Execute function creation. The platform provisions the isolated compute instance, generates an immutable Function ID (80640000000013051), initializes execution monitoring telemetry, and deploys a live REST Invocation URL endpoint under the development environment domain.

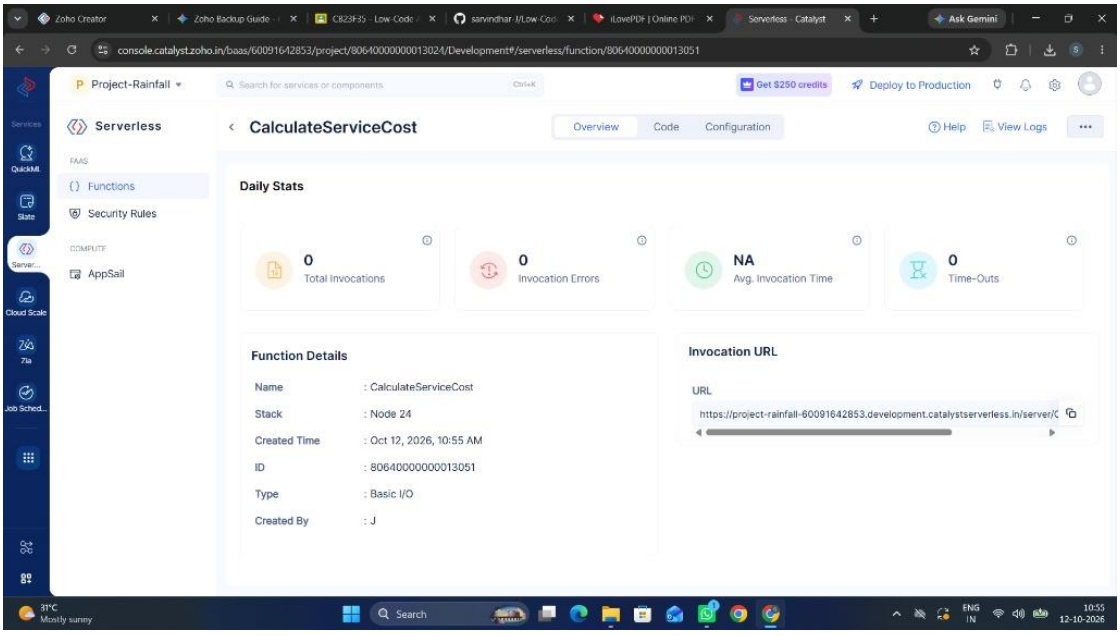


Figure 13.5: Deployed Serverless Microservice Overview, Invocation Endpoint, and Health Stats

Step 6: Bridging Low-Code Zoho Creator with Pro-Code Serverless Endpoint

To consume the Pro-Code microservice within Zoho Creator's 'Vehicle Service & Repair Request Management' system, configure an automated Deluge workflow action using the 'invokeurl' task on form submission or field validation:

```
// Deluge Workflow Script - Low-Code to Pro-Code Microservice
Integration
// Invokes the deployed Zoho Catalyst serverless endpoint:
CalculateServiceCost

catalystUrl = "https://project-rainfall-
60091642853.development.catalystserverless.in/server/CalculateServ
iceCost/execute";

// Construct payload with Vehicle Service details
payloadMap = Map();
payloadMap.put("service_type", input.Service_Type);
payloadMap.put("estimated_hours", input.Estimated_Labor_Hours);
payloadMap.put("parts_subtotal", input.Parts_Cost);

headersMap = Map();
headersMap.put("Content-Type", "application/json");

// Execute REST API Callout to Catalyst Serverless Function
response = invokeurl
[
    url: catalystUrl
    type: POST
    parameters: payloadMap.toString()
    headers: headersMap
];

// Hydrate calculated totals back into Low-Code Form fields
if(response.get("status") == "success")
{
    input.Total_Service_Cost = response.get("calculated_total");
    input.Tax_Amount = response.get("tax_component");
}
```

RESULT

Thus, the Pro-Code serverless extension for the 'Vehicle Service & Repair Request Management' system was successfully developed, deployed, and integrated using

Zoho Catalyst and Zoho Creator. The compute-intensive cost calculation logic was isolated into a scalable Node.js microservice, proving the enterprise viability of a hybrid Low-Code/Pro-Code cloud architecture.